AI CONFIGURATION & ARCHITECTURE

HORIZON GRID

AI Guide

Version: 0.1.0
Documentation prepared: 2026-08-17

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Standalone AI Backend Guide — Configuring, Testing, Switching, and Trusting the AI Layer 3

HORIZON GRID Standalone AI Backend Guide — Configuring, Testing, Switching, and Trusting the AI Layer

Why this document exists

`tech-03-ai-architecture.md` already covers the AI layer's architecture from a source-code-reading point of view — the two-call structure, the grounding/cross-check mechanisms, the `Verdict` / `RiskAssessment` data model. `user-07-ai-analysis.md` already covers what an analyst actually sees on an investigation page and how to check an AI-written claim against the real evidence behind it. This document sits alongside both: a single, hands-on guide for whoever is actually setting up and operating the AI layer — how to add credentials for each of the eleven supported backends, test a connection before trusting it, switch the active backend at runtime with zero downtime, run a side-by-side comparison between two backends on the same evidence, and understand precisely what the platform does and does not guarantee about the AI's output. Everything below is drawn directly from `backend/app/ai/service.py`, `backend/app/ai/schemas.py`, the eleven per-backend client modules (`ollama_client.py`, `anthropic_client.py`, `bedrock_client.py`, `gemini_client.py`, `groq_client.py`, `openai_client.py`, `kimi_client.py`, `deepseek_client.py`, `xai_client.py`, `mistral_client.py`, `openrouter_client.py`), `backend/app/ai/connection_test.py`, `backend/app/core/runtime_config.py`, `backend/app/core/config.py`, `backend/app/api/routes/ai_config.py`, `backend/app/api/routes/runtime.py`, and `backend/app/ai/dashboard_summary.py` — nothing here is inferred from the feature's general shape.

1. The Eleven Supported Backends

Backend	Config value	Mode	Credential(s)	Default model
Ollama	`ollama`	Local — runs on your own machine, no API key, no per-token cost	None (just a reachable base URL)	`llama3.2:3b`
Anthropic	`anthropic`	Direct API	One API key (`sk-ant-...`)	`claude-sonnet-4-5-20250929`
AWS Bedrock	`bedrock`	Cloud, via AWS	Bedrock bearer token, **or** an IAM access key + secret	`global.anthropic.claude-sonnet-4-5-20250929-v1:0`
Google Gemini	`gemini`	Cloud	One API key	`gemini-2.0-flash`
Groq	`groq`	Cloud, fast inference	One API key	`llama-3.3-70b-versatile`
OpenAI	`openai`	Direct API	One API key	`gpt-4o-mini`
Kimi (Moonshot AI)	`kimi`	Direct API	One API key	`kimi-k2.5`
DeepSeek	`deepseek`	Direct API	One API key	`deepseek-v4-flash`
xAI (Grok)	`xai`	Direct API	One API key	`grok-4.6`
Mistral AI	`mistral`	Direct API	One API key	`mistral-small-2506`
OpenRouter	`openrouter`	Cloud, meta-router across many underlying model providers	One API key	`openai/gpt-4o`

All eleven client classes expose the identical `call_claude_json(system_prompt, user_prompt, json_schema, tool_name, max_tokens)` method, which is what lets `app/ai/service.py` treat every backend interchangeably without branching on which one happens to be active. Ollama is the default specifically because it needs neither a key nor a paid quota — Bedrock needs IAM provisioning, and Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, and OpenRouter all need a cloud account with billing/quota set up; Anthropic needs only a key but still a funded account.

2. Where You Configure This

There are two places to touch AI configuration, and they serve different moments:

- **The Setup Wizard's AI Configuration page** — the first-run (or "Reconfigure") path, covered in full in `standalone-windows-installation.md`. It lets you pick a backend, enter its credentials, and test the connection before the platform ever starts. Every AI Configuration field is pre-filled with the real current values on a reconfigure run.
- **Manage Providers → AI Providers tab** (`/providers` in the running platform, requires the `provider:manage` permission — ADMIN only) — the day-to-day path for adding a second backend, rotating a key, or changing the default model without re-running the wizard. This is a real, dedicated UI tab (`frontend/app/providers/page.tsx`), separate from the **IOC Providers**, **Audit Log**, and **Network Access** tabs alongside it, and every change here "takes effect on the next investigation immediately — no restart, no editing files," per the page's own on-screen description.

Each backend appears as its own card on the AI Providers tab (`ProviderConfigRow`), showing exactly the credential fields that backend actually needs — nothing generic or one-size-fits-all:

Backend	Fields shown on its card
`ollama`	`base_url`
`anthropic`	`api_key`
`bedrock`	`bedrock_api_key`, `aws_access_key_id`, `aws_secret_access_key`, `aws_region`
`gemini`	`api_key`
`groq`	`api_key`
`openai`	`api_key`
`kimi`	`api_key`
`deepseek`	`api_key`
`xai`	`api_key`
`mistral`	`api_key`
`openrouter`	`api_key`

Every card also carries a model-ID field, a **Test Connection** button, a **Save** button, and — for whichever backend isn't already active — a **Set Active** button. A backend that is currently active shows an "Active" badge instead of the Set Active button.

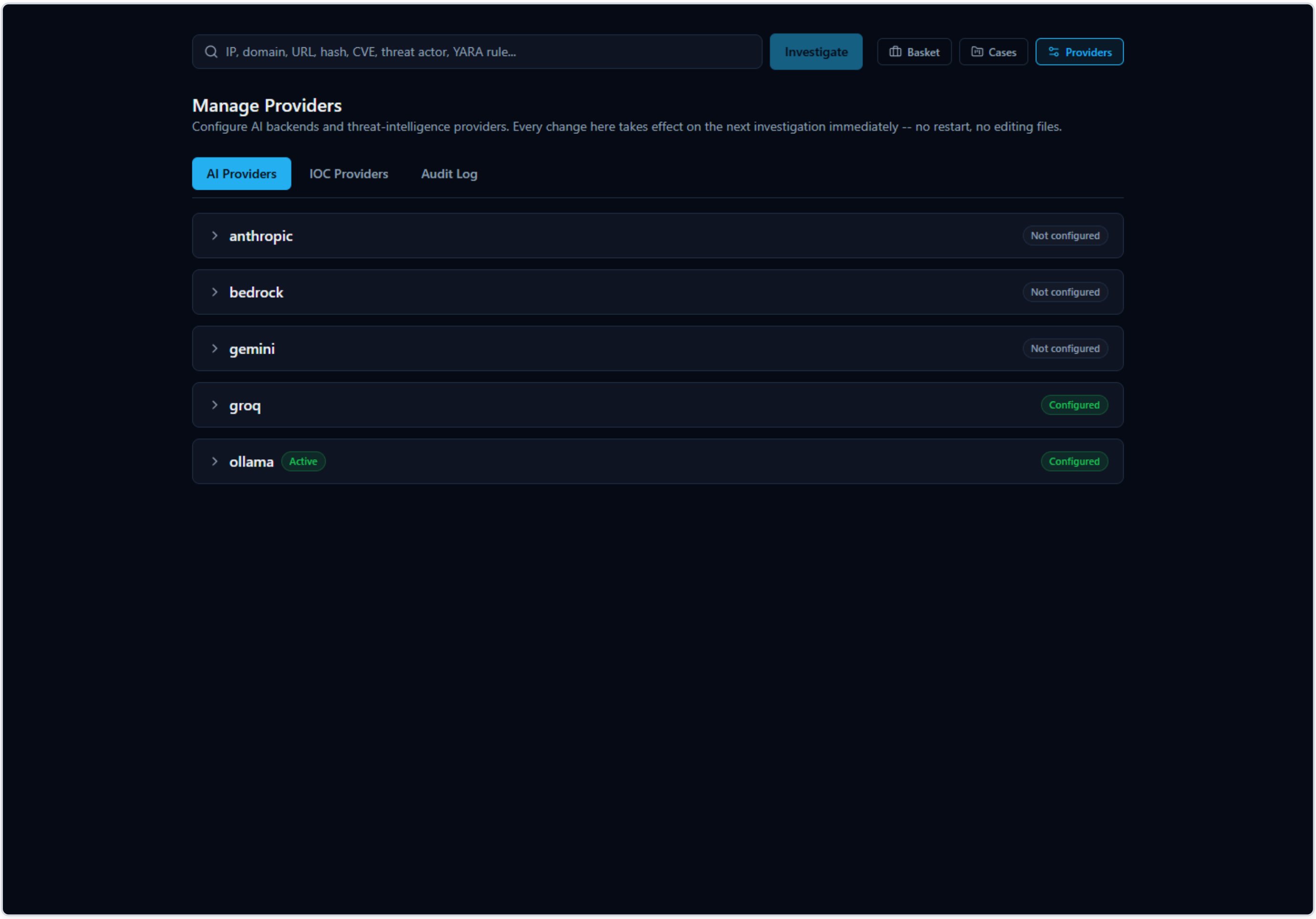

Figure 1 — The Manage Providers page's AI Providers tab, showing all eleven backend cards with their credential fields, Test Connection/Save buttons, and the Active badge on whichever backend is currently selected.

3. Adding and Configuring Each Backend

Ollama (local)

Field: `base_url` (model is a separate field, not a credential). The backend process runs inside Docker while Ollama itself normally runs directly on the host machine, so the base URL is **not** `http://localhost:11434` from the container's point of view — it defaults to `http://host.docker.internal:11434`, which Docker Desktop on Windows resolves to the host automatically. If Ollama runs somewhere else on your network, point this at that host instead. There is nothing to redact here: Ollama needs no API key at all.

Before pointing the platform at a model, that model has to actually be pulled on the Ollama instance (`ollama pull llama3.2:3b` or whichever model you intend to use) — the connection test below will tell you plainly if it isn't.

Anthropic (direct API)

Field: `api_key` (a real `sk-ant-...` key from the Anthropic Console). This is the simplest cloud backend to provision: unlike Bedrock, there's no IAM policy or model-access grant to request, and unlike Gemini, there's no separate cloud project to create — the key alone, on a funded account, is sufficient.

AWS Bedrock

Bedrock supports two independent auth schemes, and you only need one:

- **Bedrock API key (bearer token)** — the `bedrock_api_key` field, populated from AWS IAM's own "Generate API key" button. This is the preferred path: the platform sets `AWS_BEARER_TOKEN_BEDROCK` from this value and botocore's own token-provider chain picks it up automatically.
- **Classic IAM access key + secret** — `aws_access_key_id` + `aws_secret_access_key` (plus `aws_region`, defaulting to `us-east-1`), for an account that grants `bedrock:InvokeModel` through a normal IAM user or role instead of a bearer token.

Whichever scheme you use, the account/role needs actual model access granted for the configured `model_id` in that AWS region — a correctly-formed credential with no model access will fail the connection test with a specific "model not found or not enabled in this account/region" message rather than a generic authentication error, so read the test result's exact wording before assuming the key itself is wrong.

Google Gemini

Field: `api_key`, from a Google AI Studio / Cloud project with the Gemini API enabled and billing configured. The key is sent via the `x-goog-api-key` header rather than a `?key=` query parameter — deliberately, since request URLs (including query strings) are what get written to plain HTTP logs, and a query-string key would otherwise land in those logs on every call.

Groq

Field: `api_key`, from the Groq console (`console.groq.com`). Groq is a distinct inference provider from "Grok" (xAI) — this is `api.groq.com`, an OpenAI-compatible chat-completions API.

Groq's model field is the first one in this guide that behaves differently from Ollama, Anthropic, Bedrock, and Gemini above: instead of a fixed dropdown, the platform discovers Groq's real, currently-available model list **live**, by calling Groq's own `GET /v1/models` with whatever key you've just typed (`POST /api/v1/ai/groq/models` on the backend, wired to the wizard/UI's model field). This exists because Groq's hosted model lineup changes over time — a hardcoded list would silently go stale as Groq deprecates or introduces models. If no key has been entered yet, or the live call fails, the field falls back to a short static list (`llama-3.3-70b-versatile`, `llama-3.1-8b-instant`, `openai/gpt-oss-120b`) that is explicitly not treated as exhaustive — whatever model ID you actually submit to a real chat-completion call is sent as-is; Groq's own API is the only authority on whether it exists, not this fallback list. Every backend documented after this one (OpenAI, Kimi, DeepSeek, xAI, Mistral, and OpenRouter) uses this same live-discovery mechanism for the identical reason.

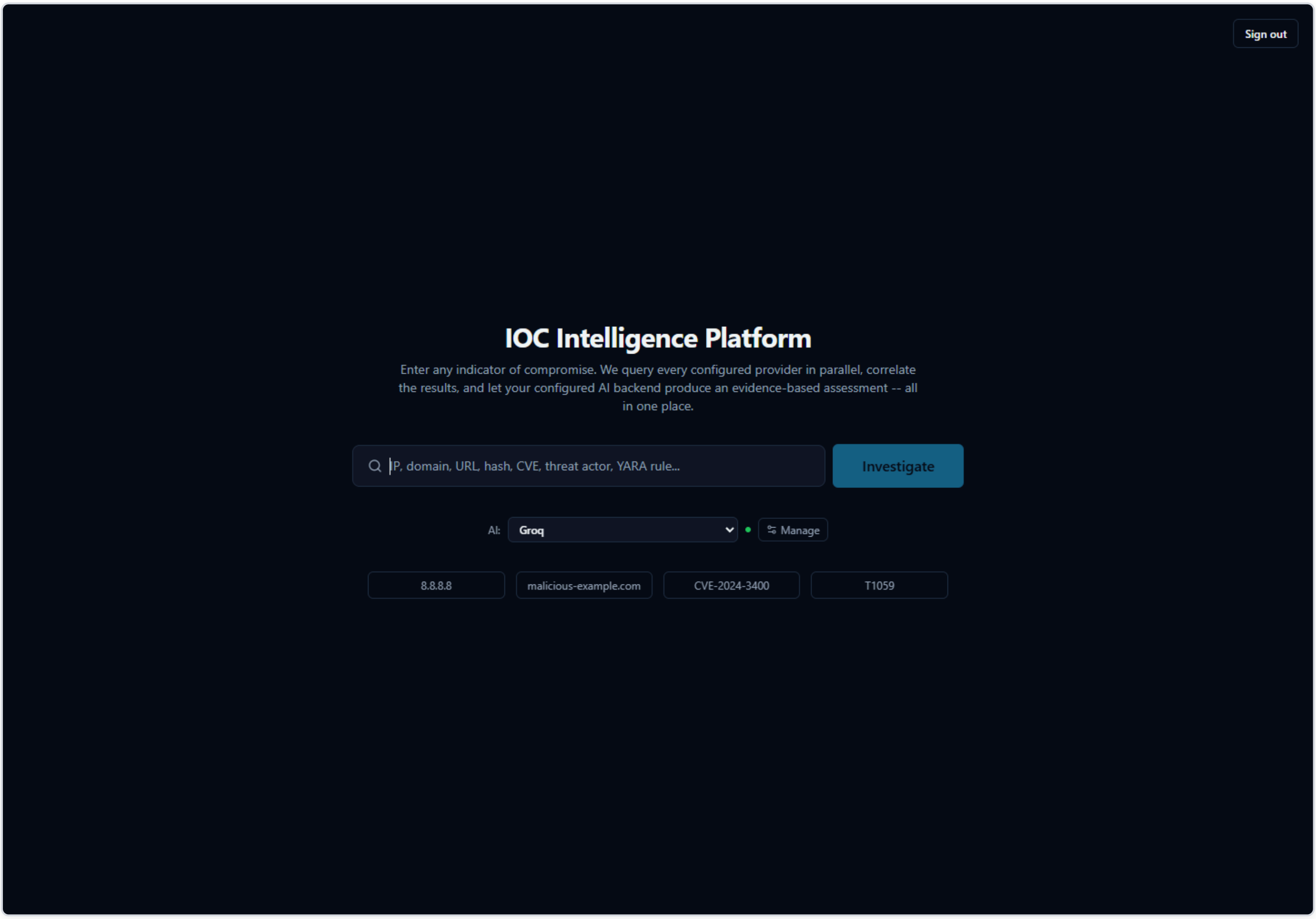

Figure 2 — The Groq AI Providers card with its live-refreshing model dropdown after a valid API key has been entered.

OpenAI

Field: `api_key`, from `platform.openai.com/api-keys`. Base URL `https://api.openai.com/v1`, default model `gpt-4o-mini`. OpenAI is, architecturally, the template every other OpenAI-*compatible* backend in this module (Groq, Kimi, DeepSeek, xAI, Mistral, OpenRouter) copies its request/response shape from — `openai_client.py`'s own docstring says this outright: "OpenAI is the origin of the request/response shape both clients use, not a coincidence."

Like Groq, OpenAI's model field is live-discovered rather than a fixed dropdown (`POST /api/v1/ai/openai/models`, calling OpenAI's own `GET /v1/models`). OpenAI's `/models` endpoint lists every model visible to the account, including embedding, moderation, image, audio, and realtime/transcription models that can't serve a chat-completion call at all — `list_models()` filters these out, keeping only ids that start with `gpt-`, `o1`, `o3`, `o4`, or `chatgpt-` and excluding anything with `audio`, `realtime`, `transcribe`, `tts`, or `embedding` in the id, so the dropdown isn't cluttered with entries that would fail immediately if selected. If that filter happens to remove every candidate, `list_models()` falls back to returning the full unfiltered id list rather than an empty dropdown.

Kimi (Moonshot AI)

Field: `api_key`, from `platform.moonshot.ai/console/api-keys`. Base URL `https://api.moonshot.ai/v1`, default model `kimi-k2.5`.

The default model choice here is deliberate, not arbitrary: this platform's structured output relies on forcing a specific `tool_choice` on every call, and Moonshot's newer "thinking"-mode models — `kimi-k3` and `kimi-k2.7-code` — always have thinking mode on with no way to disable it, which makes a forced `tool_choice` call fail with an HTTP 400. `kimi-k2.5` and the `moonshot-v1-*` family have no thinking parameter at all and work with forced tool-calling with zero special handling, which is why `DEFAULT_MODEL` is pinned to `kimi-k2.5` rather than a newer model. `kimi_client.py`'s own comments note that `kimi-k2.6` can go either way depending on how it's deployed — if a forced-tool-call 400 shows up against `kimi-k2.6` or a future thinking-capable model, the code-level fix is adding a `"thinking": {"type": "disabled"}` field to the request body for that model, deliberately not done pre-emptively so this client stays as simple/uniform as the others. The connection test surfaces this distinctly: a `400` response whose body mentions "thinking" is reported as "Model '...' requires disabling 'thinking' mode to use a forced tool call — pick a non-reasoning model (e.g. kimi-k2.5) or a model where thinking can be disabled," rather than a generic failure.

Kimi's model field is also live-discovered (`POST /api/v1/ai/kimi/models`, against Moonshot's own `GET /v1/models`); unlike OpenAI's, Moonshot's listing doesn't distinguish chat-capable models from any other kind, so there's nothing for `list_models()` to filter there.

DeepSeek

Field: `api_key`, from `platform.deepseek.com/api_keys`. Default model `deepseek-v4-flash`.

The one real, verified quirk worth knowing before typing a base URL by hand: DeepSeek's base URL is `https://api.deepseek.com` — **no** `/v1` segment — unlike every other OpenAI-compatible backend in this module (Groq's `https://api.groq.com/openai/v1`, OpenAI's own `https://api.openai.com/v1`, and so on). `deepseek_client.py`'s docstring is explicit that this was "confirmed verbatim from live docs," not a typo left uncorrected.

DeepSeek's model field is live-discovered the same way (`POST /api/v1/ai/deepseek/models`, against `GET https://api.deepseek.com/models`); every model DeepSeek's own listing returns is chat-capable, so there's no filtering to do. One other DeepSeek-specific behavior: its API returns a provider-specific HTTP `402` ("Insufficient Balance") in addition to the standard 401/404/429 codes, and the connection test reports that distinctly as "DeepSeek account balance is insufficient to make this request" rather than a generic failure.

xAI (Grok)

Field: `api_key`, from `console.x.ai`. Base URL `https://api.x.ai/v1`, default model `grok-4.6`.

xAI is a **different product from Groq** (`api.groq.com`) — the same naming-collision warning the Groq section above makes in the other direction. `xai_client.py`'s own docstring calls this out explicitly, and the connection test's authentication-failure message spells out "xAI (Grok)" rather than a bare "xAI" for exactly this reason. One xAI-specific quirk worth knowing: unlike every other backend's connection test in this module, xAI's check treats an HTTP `400` the same as a `401` — both are reported as "Authentication failed — check your xAI (Grok) API key" — because xAI's own error-response shape is flat (`{"code": "...", "error": "message text"}`, a bare top-level `error` *string*) rather than OpenAI's nested `{"error": {"message": ...}}` object, confirmed live against `api.x.ai`.

xAI's model field is live-discovered the same way as the others (`POST /api/v1/ai/xai/models`). `xai_client.py` also notes that xAI's own documentation states forced `tool_choice` "should" be followed by the model but is "not enforced at the moment" — there's no guaranteed strict schema adherence on xAI's end, which is why the client's JSON-decode error handling on the tool-call arguments is left exactly as defensive as every other backend's, not loosened.

Mistral AI

Field: `api_key`, from `console.mistral.ai/api-keys`. Base URL `https://api.mistral.ai/v1`, default model `mistral-small-2506`.

Mistral versions its models with dated suffixes (`mistral-small-2506`, `mistral-large-2411`) rather than a single rolling name, though rolling aliases such as `mistral-large-latest` also exist on Mistral's side; `DEFAULT_MODEL` follows this project's existing convention of defaulting to the smaller/faster/cheaper model rather than the largest (the same reasoning behind OpenAI's `gpt-4o-mini` and Groq's `llama-3.3-70b-versatile` defaults). Mistral's model field is live-discovered the same way as the other cloud backends (`POST /api/v1/ai/mistral/models`).

Unlike DeepSeek and OpenRouter, Mistral's error-response body shape wasn't confirmed against a dedicated schema during this client's own research — `mistral_client.py`'s docstring notes this plainly rather than guessing — so, same as the OpenAI-template default, a non-2xx response is surfaced as raw response text rather than parsed for a provider-specific field.

OpenRouter

Field: `api_key`, from `openrouter.ai/keys`. Base URL `https://openrouter.ai/api/v1`, default model `openai/gpt-4o`.

OpenRouter is architecturally different from every other backend in this section: it is not a model provider of its own, but a **meta-router** giving access to hundreds of underlying models from many different companies (OpenAI, Anthropic, Google, Meta, DeepSeek, and others) through one OpenAI-compatible API surface. That has a direct, load-bearing consequence for this platform: most of those underlying models do **not** support the forced `tool_choice` structured-output mechanism this app requires for every AI call, and selecting one that doesn't would 400 against `call_claude_json()`.

`list_models()` handles this by filtering OpenRouter's live `GET /models` response down to ids whose own `supported_parameters` array includes `"tool_choice"` — confirmed live at the time `openrouter_client.py` was written, when 342 of 413 listed models declared `tool_choice` support — so the wizard/UI's model dropdown (`POST /api/v1/ai/openrouter/models`) never offers a model that would fail immediately. If a future response happens to omit `supported_parameters` for every model (nothing left to filter on), `list_models()` falls back to returning every listed id rather than an empty dropdown. `FALLBACK_MODELS` mirrors the same constraint with a short, static list of current, tool-calling-capable ids spanning several underlying providers (`openai/gpt-4o`, `anthropic/claude-sonnet-4.5`, `google/gemini-2.5-pro`, `deepseek/deepseek-chat`, `meta-llama/llama-3.3-70b-instruct`), used only when a live call isn't possible. OpenRouter also returns an HTTP `402` for insufficient account credits, reported distinctly by the connection test as "OpenRouter account has insufficient credits for this request."

[MISSING FIGURE: standalone-ai-openrouter-model-dropdown.png]

4. Testing a Connection Before You Trust It

Every one of the eleven backends has a real, live "does this actually work" check — `POST /api/v1/ai/test` (`app/api/routes/ai_config.py`'s `ai_test_connection()`, calling `app/ai/connection_test.py`'s `test_ai_connection()`), gated behind the same `provider:manage` permission as everything else on this page. Two properties hold for every backend, without exception:

- **It always tests the candidate credentials you just typed, never whatever is already saved.** This is a deliberate trust-model choice, shared with the equivalent IOC-provider test: you can safely try a new key in the field before overwriting a working one, and testing never mutates the stored configuration.
- **It makes one real, minimal round trip to the backend's own API** — a one-word "reply with exactly one word: pong" prompt — and reports what actually happened: whether it succeeded, which model actually replied, and the real measured latency in milliseconds. Nothing about the result (model name, latency, success) is simulated.

The failure messages are specific rather than a generic "failed," because each backend's real HTTP status codes are interpreted individually: a `401`/`403` reports as an authentication problem ("check your API key"/"check your credentials and IAM permissions"), a `404` reports as the specific model ID not being found or not enabled on that account, and a `429` reports as a rate limit with a note that the key itself may still be valid. Ollama's own connection test additionally refuses to test an unsafe outbound URL (`assert_safe_outbound_url`) and, on a `404`, tells you directly to `ollama pull <model>` first rather than leaving you to guess why a locally-hosted model didn't respond.

Clicking **Save** after a successful test persists the credential into the runtime-mutable configuration store (not just `.env`); a separate `record-test` call then stores that result so the AI Providers tab can show "last tested: ok, N minutes ago" the next time you open it, without silently re-testing on every page load.

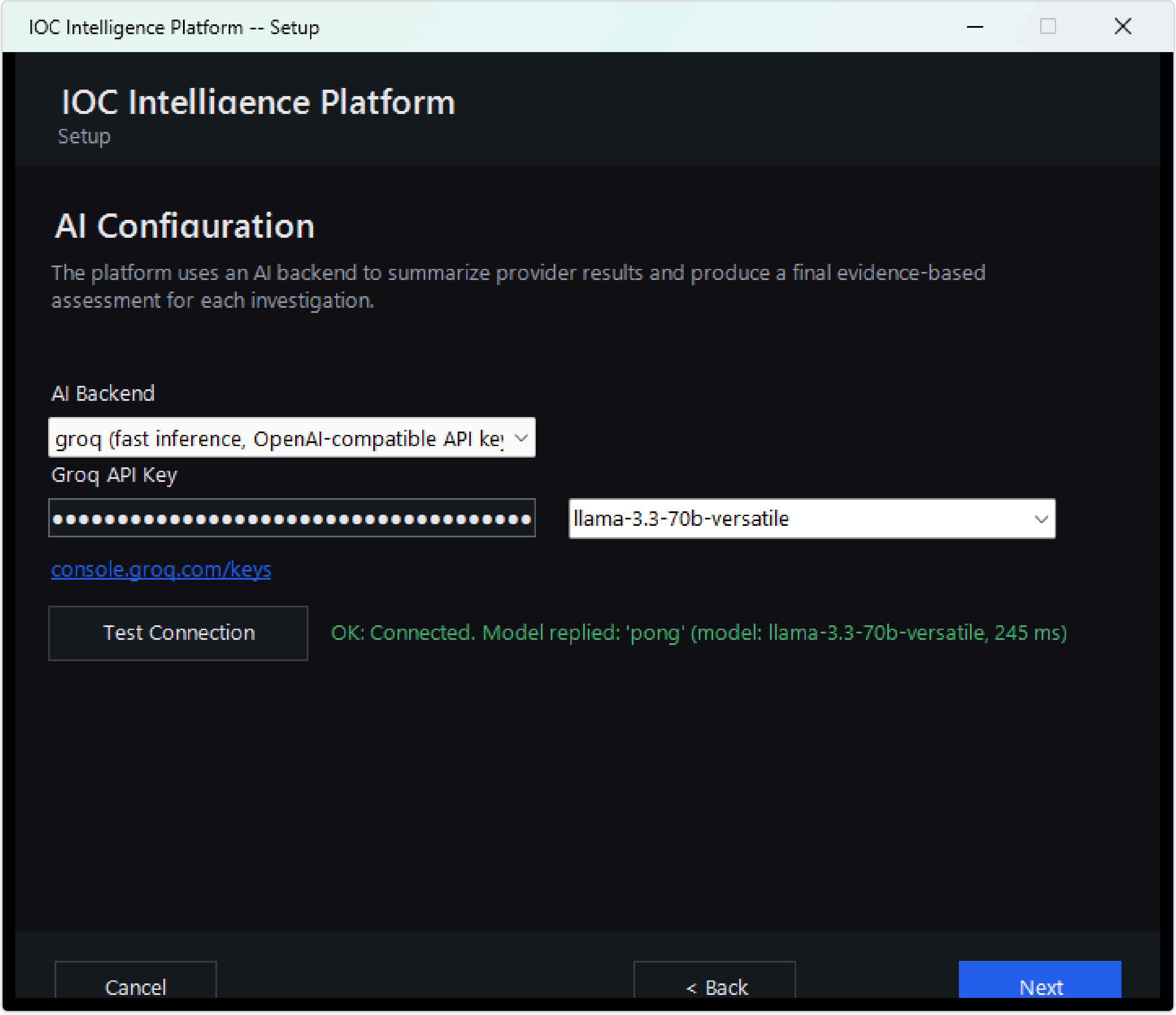

Figure 4 — The Anthropic AI Provider card immediately after a successful Test Connection click, showing the real measured latency and the model that replied.

5. Switching the Active Backend at Runtime — Zero Downtime

Every backend's credentials and model ID can be saved without making that backend active — configuring Groq doesn't stop Ollama (or whichever backend is currently active) from continuing to serve every AI call until you explicitly switch.

Switching is one action: clicking **Set Active** on a backend's card in the AI Providers tab, or picking a different entry from the compact **"AI: [dropdown]"** control next to the home page's search bar (`AiQuickSwitch.tsx`) — both call the identical `POST /api/v1/runtime/ai-active`. This persists which backend row in the `provider_runtime_configs` table has `is_active = true`, and the effect is immediate: **the very next AI call anywhere in the platform** — the next per-provider summary or final assessment, for any in-flight or brand-new lookup, and the next Dashboard executive-summary generation — uses the newly active backend and its credentials. There is no container restart, no `.env` edit, and no redeploy involved.

What makes this possible mechanically is that `app/ai/service.py`'s `_get_ai_client()` resolves which backend/credentials to use **on every single call**, not once at process start. The resolution order is:

1. An explicit `backend_override`, used only by the AI-comparison feature (§6 below) to name a specific non-active backend for one comparison run, without changing the platform-wide active backend.
2. The currently **active** runtime-configured backend — a fresh database read of `provider_runtime_configs` on that exact call.
3. The legacy, frozen `Settings` singleton (`.env`-derived), kept only as a fallback for an install where no runtime config has ever been seeded at all.

Because step 2 is a fresh read every time rather than a cached value from process start, and because each of the eleven client classes accepts constructor overrides so a fresh, correctly-credentialed instance can be built per call instead of reusing a stale singleton, "switch AI backend" really is a config change with immediate effect, not a code change or a restart.

If the resolved backend's `is_configured` check fails (a backend selected active with no working key/URL), the platform raises immediately with a clear message naming the problem, rather than letting the request fall into a confusing low-level failure — e.g. an `httpx` `TypeError` on a `None` header, or Gemini silently sending the literal string `"None"` as an API key and getting back an ordinary-looking 4xx.

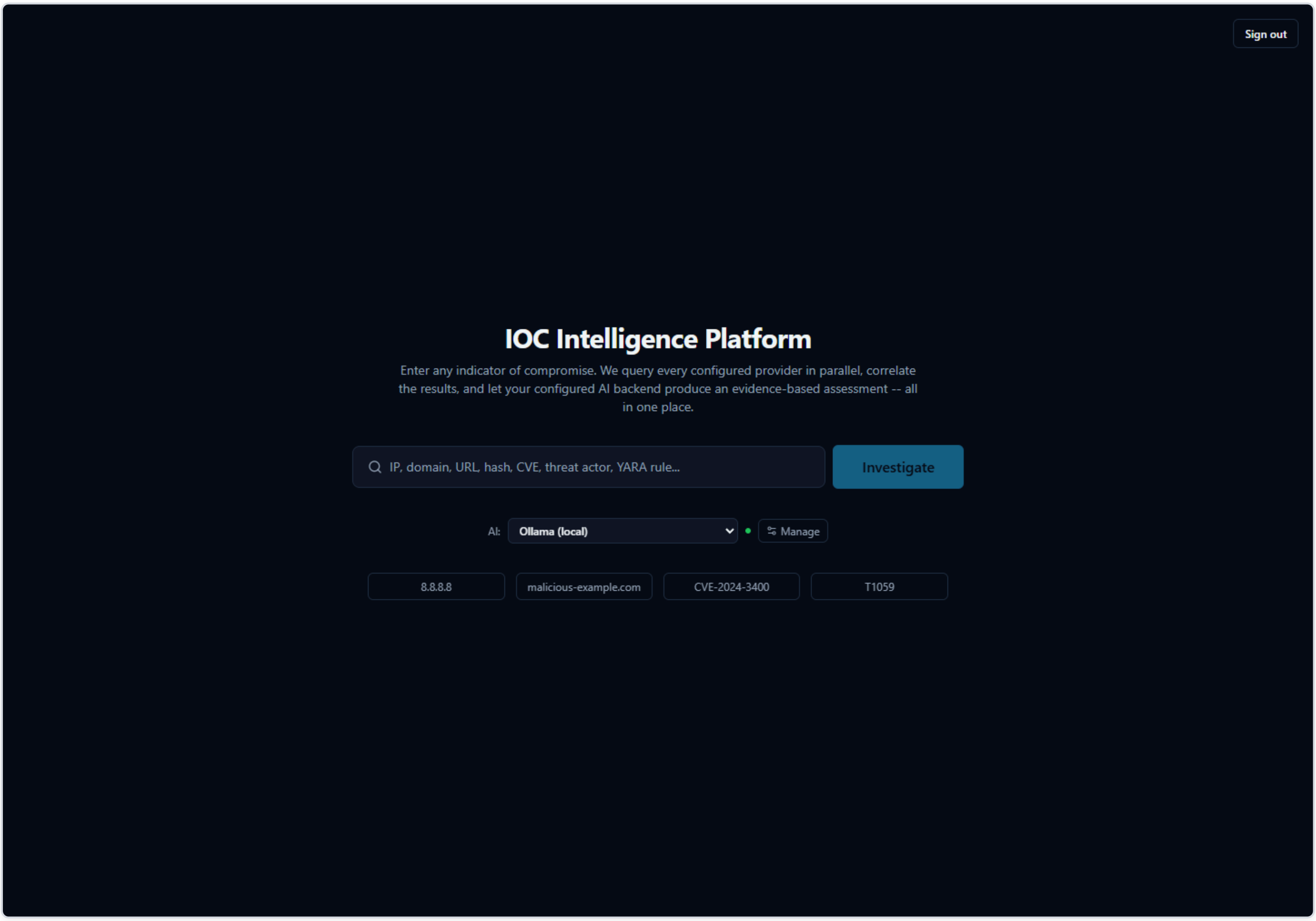

Figure 5 — The home page's "AI:" dropdown open, showing all eleven backends with their configured-status dot, immediately before switching the active backend.

6. AI Backend Comparison — Reanalyzing the Same Evidence

Because the active backend can change between one lookup and the next, an analyst can also deliberately ask "what would a *different* backend have concluded from this exact same evidence?" without re-running the whole investigation. This is the **AI Comparison** panel on a completed investigation's page (`AiComparisonPanel.tsx`), and it maps directly to `POST /api/v1/lookup/{lookup_id}/reanalyze`.

A reanalysis run:

- **Never re-queries any IOC provider.** It rebuilds `provider_summaries` from the already-persisted `AISummaryRecord` rows and rebuilds the correlation result purely from already-persisted correlation-edge rows — the exact same reconstruction the original investigation's own final-assessment step used, just replayed from storage instead of freshly computed.
- **Recomputes the deterministic score from the same persisted evidence** (provider results, correlation, and any Security Assessment Toolkit findings — including findings added to this lookup *after* the original investigation ran) rather than reusing whatever score the original assessment happened to persist. This is deliberate: every backend compared side-by-side against the same evidence must show the **identical** `overall_risk_score`/`confidence_score`/`malicious_probability`/`severity`, differing only in AI-authored prose and verdict choice among the options those numbers actually support. A backend that appeared to "compute" a different score would defeat the entire point of a same-evidence comparison.
- **Never overwrites the original assessment.** Every result — the original plus every later comparison — is written as its own row in `final_assessment_records`, tagged with the `ai_backend`/`ai_model` that produced it and an `is_primary` flag distinguishing the original from a comparison. `GET /api/v1/lookup/{lookup_id}/assessments` returns the full set for side-by-side review.

In the UI, picking a backend from the dropdown and clicking **"Analyze with `<backend>`"** appends a new card to the list, each one labeled "Original" or "Comparison" alongside its backend/model, its verdict badge, and its risk/confidence/malicious-probability numbers. Neither result is presented as more authoritative than the other — the value is in the second opinion itself, and in the ability to see two backends actually agree (or disagree) on the same underlying evidence.

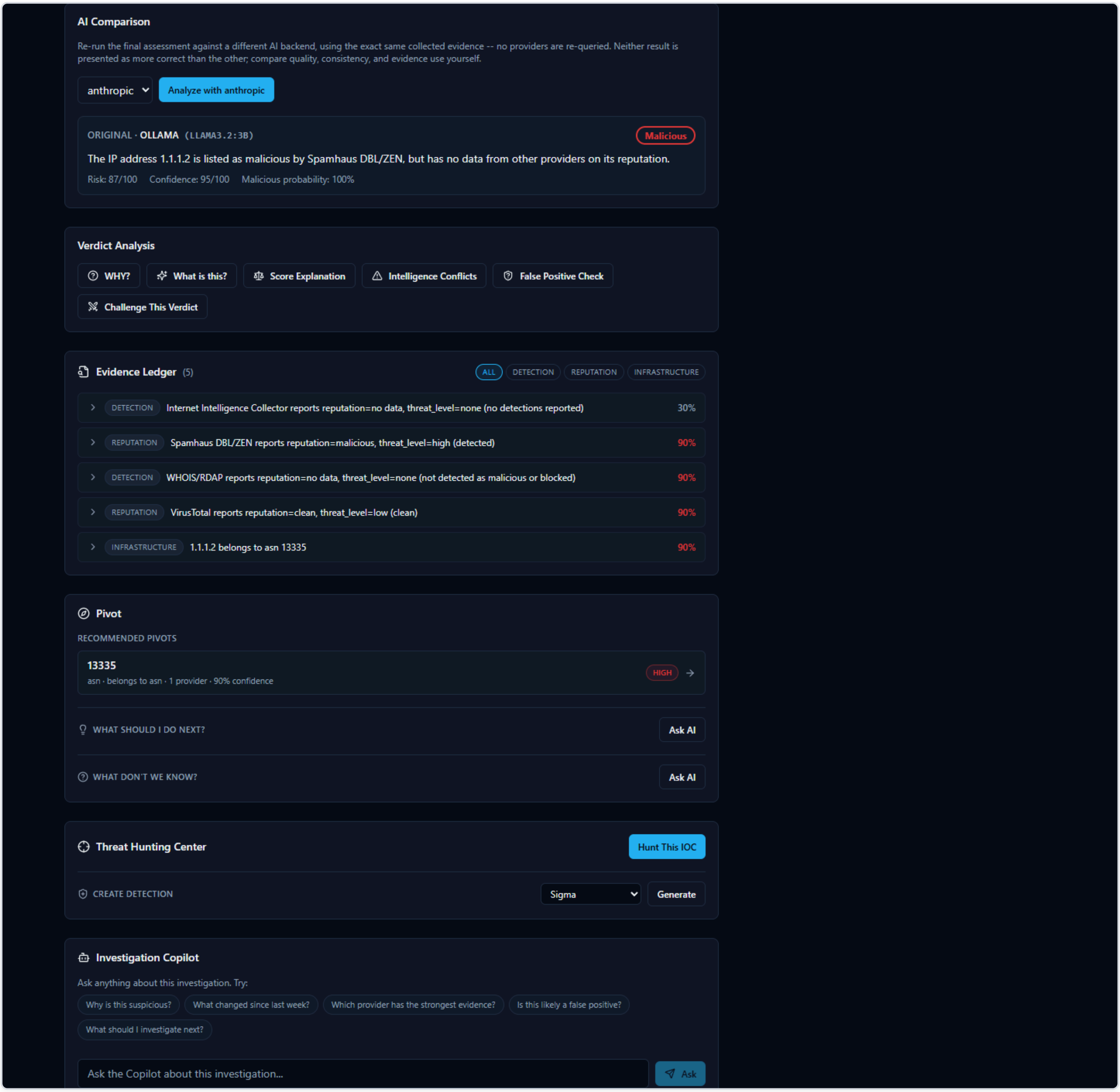

Figure 6 — The AI Comparison panel on a completed investigation, showing the original assessment and a "Comparison" result from a second backend, with identical risk-score numbers but different executive-summary prose.

7. The Three AI Outcomes — And Why "Skipped" Is Not a Failure

Every `FinalAssessment` this platform ever produces carries an explicit `ai_outcome` field, one of exactly three values (`app/ai/schemas.py`'s `_AIOutcome`), set at all three of `generate_final_assessment()`'s return points in `service.py`:

`ai_outcome`	What it means	When it happens
`success`	The AI backend was called, its structured output validated, and the response was returned.	The normal path.
`skipped_no_evidence`	A **correct decision** never to call the AI at all — there was nothing to analyze.	`provider_summaries` is empty **and** the correlation engine found zero edges.
`failed`	A **genuine generation failure** — the AI was called but every retry attempt still raised.	E.g. a rate-limited or unreachable backend, or output that never validated.

The distinction between `skipped_no_evidence` and `failed` exists specifically so a later reliability metric — the Dashboard's `ai_success_rate` KPI — can tell "there was genuinely nothing to analyze" apart from "the AI was asked and it broke," rather than lumping both into one number. `ai_success_rate` explicitly excludes `skipped_no_evidence` from **both** the numerator and the denominator of its calculation: an investigation where no provider returned any data is not a strike against the AI's reliability, and counting it as one would make a platform with excellent AI reliability look artificially worse purely because some investigations legitimately had zero evidence to work with.

The `skipped_no_evidence` path exists because of a real, reproduced failure, not a hypothetical one: looking up the EICAR antivirus test file's actual MD5 hash (`44d88612fea8a8f36de82e1278abb02f`, a standard, harmless, industry-wide AV test file) with every relevant provider left unconfigured — meaning zero real evidence of any kind reached the prompt — still caused a small local model (`llama3.2:3b`) to return `final_verdict="highly_malicious"` and `malicious_probability=92`, fabricating an "association with ransomware and trojans" wholesale from its own pretrained knowledge of a famous hash, directly violating its own system-prompt instruction never to fabricate. No prompt wording fixed this reliably — the model already had that instruction and ignored it — so the platform doesn't call the AI at all whenever `provider_summaries` and `correlation.edges` are both empty: an empty-evidence case has exactly one correct answer (`unknown`, all risk fields at their real deterministic value, an honest "no provider returned usable data" message) regardless of which backend happens to be configured, so deterministic code produces it instead of hoping a model declines to guess.

8. Prompt Architecture, at a High Level

`tech-03-ai-architecture.md` §3 already covers the two structurally distinct call types in detail — `summarize_provider()` (one call per provider that returned real data, grounded only in that provider's own JSON) and `generate_final_assessment()` (one call per lookup, grounded in every provider summary plus the correlation engine's output, never in raw provider JSON again) — along with the grounding/cross-check mechanisms that filter the model's `agreeing_providers`/`disagreeing_providers` claims and flag ungrounded MITRE mappings. That material isn't repeated here; this section adds one mechanism that document doesn't cover: how a single provider's raw payload is actually shaped before it reaches `summarize_provider()`'s prompt at all.

Field-length pruning (`_prune_for_prompt()`)

The module's own docstring states that raw provider JSON should never reach a prompt unbounded — this is the one call site (`summarize_provider()`, not `generate_final_assessment()`, which is already bounded by construction since it only ever sees already-distilled summaries) where that rule has to be actively enforced, because a single provider's raw response can be enormous.

This was tuned against two real, reproduced failures, not designed abstractly:

1. **NIST NVD's `configurations` field.** For a genuinely critical CVE (CVE-2021-44228, Log4Shell), this field runs to hundreds of nested CPE-match entries — dozens of times longer than the actual signal (`verdict`, `cvss_score`, `cvss_severity`, `description`). Feeding it to the model unbounded buried the CRITICAL severity under noise the model's attention gravitated to instead, reproduced on two different backends (a local 3B model and a hosted 70B model), both calling a CVSS 10.0 remote-code-execution vulnerability "benign" or "unknown risk."

2. **A single uniform cap wasn't the fix.** A flat 800-character cap fixed NVD but broke MITRE ATT&CK, whose `description` field is that provider's *only* signal-bearing field (no separate short verdict/score field the way NVD has) and routinely runs 600–1,800+ characters for a real technique — the cap silently chopped legitimate signal mid-sentence, the same "AI loses the signal" failure this function exists to prevent, just from the opposite direction. Raising the same uniform cap to 4,000 characters fixed MITRE but then broke NVD a second way: NVD's `references` field (a bare list of URLs, 7,851 characters observed) and `configurations` (67,721 characters observed) both blew past a per-field cap of 4,000 combined, pushing a real Groq call to an HTTP 413 against Groq's per-minute token limit.

The actual fix distinguishes **why** a field is long, not just how long: a `str` field (prose, like MITRE's `description`) gets a generous 2,000-character cap, since long prose is far more likely to be genuine single-field signal; a `list` / `dict` field (inherently structural or repetitive — CPE match entries, URL lists, raw report objects) gets a tighter 800-character cap regardless of provider, since every oversized list/dict field examined across both NVD and AbuseIPDB turned out to be low-signal bulk, never the primary carrier of a verdict. Short, scalar fields — the ones actually carrying a verdict — pass through untouched either way, and a truncated field is marked as truncated (with its real original length) rather than silently cut.

9. The Deterministic-Score Lock — Why the AI Can Narrate a Score but Never Set One

This is the single most load-bearing design fact in this layer, and it exists because of directly observed failures, not as a theoretical safeguard.

`app/scoring/engine.py` computes `overall_risk_score`, `confidence_score`, `malicious_probability`, and `severity` **before any AI call happens at all**, from the deterministic combination of provider-verdict consensus and correlation-graph evidence. By the time `generate_final_assessment()` runs, those four numbers already exist and are already correct — the AI's job for this part of its output is narration and justification, never computation.

The user prompt hands the model those four values in an explicit "Deterministic risk assessment (already computed — GIVEN, do not recalculate)" block, and the system prompt instructs the model to echo them back verbatim into the `risk` object and choose a `final_verdict` consistent with them — never to invent different numbers, and never to let a verdict contradict them. That instruction alone, however, is **not** what actually enforces this — testing showed prompt wording by itself was not reliable enough, especially against a small local model:

- `RiskAssessment`'s own field validator (`_reject_0_to_1_scale`) had to be added because `llama3.2:3b` was directly observed defaulting to a 0–1 probability convention (returning `0.5` for "50%") despite the field's explicit 0–100 description.
- `FinalAssessment`'s `_verdict_must_agree_with_risk` validator had to be added because the same small model was directly observed emitting self-contradictory pairs in a single response — e.g. `final_verdict="malicious"` alongside a `malicious_probability` around 10–20 — that pass each individual field's own validation but make no sense together.

So the platform does not rely on the model getting this right. After a response validates, `generate_final_assessment()` unconditionally **overwrites** `risk.overall_risk_score`, `confidence_score`, `malicious_probability`, `severity`, `scoring_engine_version`, and `scoring_breakdown` with the real, already-computed values from the scoring engine — regardless of what the model actually emitted for those fields. Critically, it does not stop at swapping the numbers in: it re-runs `FinalAssessment.model_validate()` on the **entire object** with the real risk spliced in, which re-executes every validator on the model — including `_verdict_must_agree_with_risk` — against the values that are actually about to be persisted, not just against whatever the model itself originally emitted. A model that emitted a self-consistent-but-wrong pair (its own invented `malicious_probability=92` alongside `final_verdict="malicious"`) would sail through the validator on its own numbers; only re-validating against the *real* numbers can catch a verdict that flatly contradicts them. If that re-validation fails, the same retry loop that already exists for a first-pass validation failure gives the model one more attempt, still working from the same given numbers. `ai_backend` and `ai_model` are overwritten the same mechanical way, with the real, code-derived identity of whichever backend actually produced the call — never left to a model's own (possibly wrong, possibly absent) self-report.

The practical upshot: re-analyzing the same evidence with a different backend (§6) will always show the identical score, every time, no matter which of the eleven backends produced it or how that backend chose to phrase its reasoning — because the score was never the AI's to decide in the first place.

10. The Executive Summary's Honest AI/Template Disclosure

The Dashboard's AI-generated executive narrative (`GET /api/v1/dashboard/executive-summary`, `app/ai/dashboard_summary.py`) follows the identical discipline as §9, applied to KPI numbers instead of a risk score: `app/core/dashboard.py`'s `get_kpis()` is the **only** source of truth for every number the narrative states, and the AI is handed those seven KPI values as given facts it must not recalculate, round, or restate differently.

If the AI call fails outright, or its output fails validation on both the original attempt and its one retry, the endpoint does not error and does not return a blank summary — it falls back to `_template_fallback_narrative()`, a deterministic, template-built sentence constructed directly from the same real KPI numbers (e.g. "There are currently N active investigation(s) in progress and N open case(s)..."). Whichever path actually produced the text, the response's `source` field honestly says so — `"ai"` or `"template_fallback"` — rather than letting a fallback silently look like a real AI success. The Dashboard's Executive Summary card surfaces this directly as a visible badge reading **"AI-generated"** or **"Template fallback"** next to the narrative, so nobody reading it mistakes a template sentence for an AI-authored one.

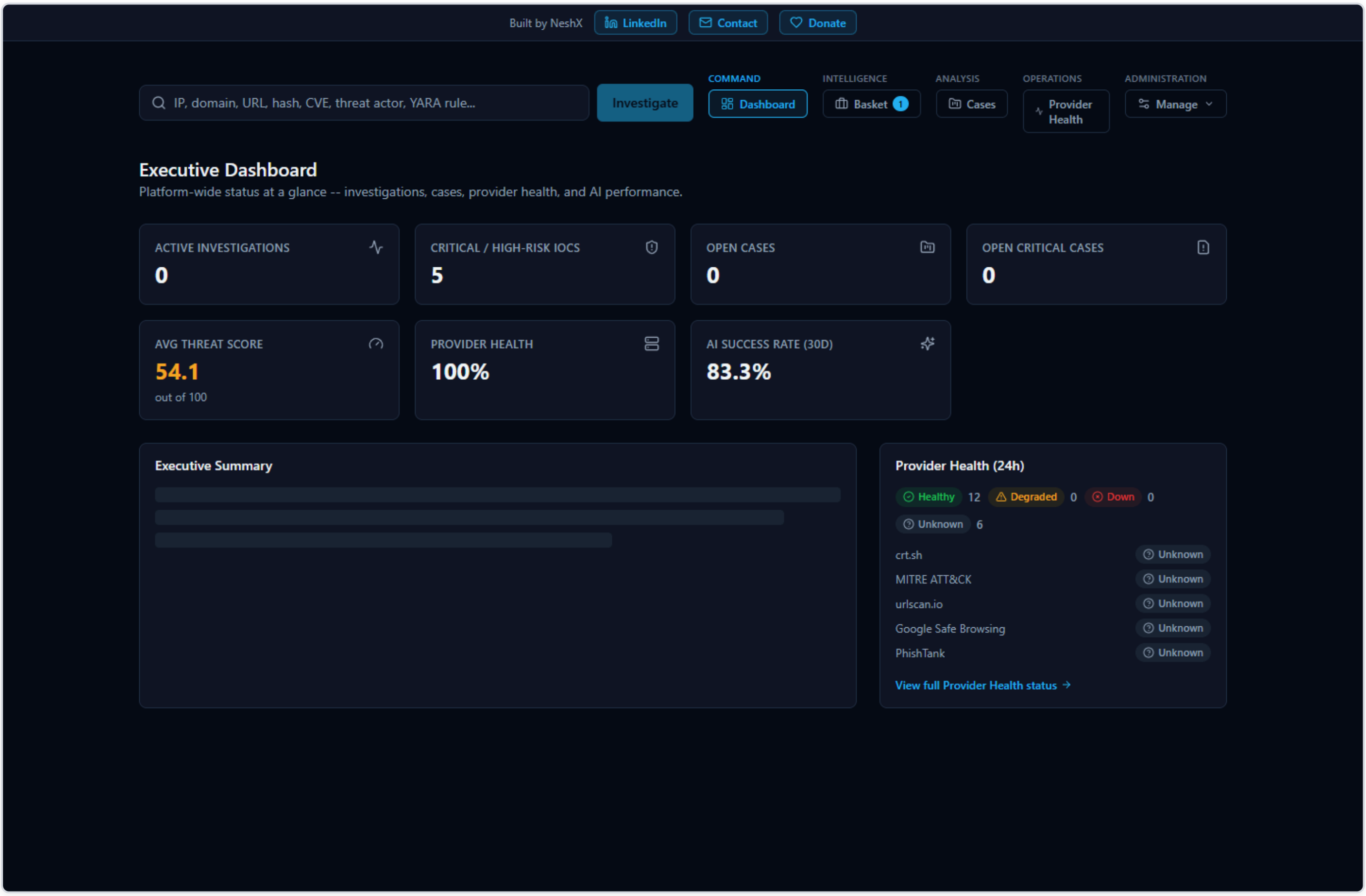

Figure 7 — The Dashboard's Executive Summary card, showing the narrative text alongside its "AI-generated" or "Template fallback" source badge.

11. Known Limitation: A Local Ollama Backend Serializes Concurrent Requests

This is a real, tested, and already-documented characteristic (also noted in `tech-09-performance.md` and the project's `FINAL_RELEASE_QA_REPORT.md`), not a newly discovered issue: when the platform is configured to use a local Ollama model, that single local model processes generation requests **one at a time**. Under light or moderate use this is unnoticeable, but under heavy *concurrent* load — for example, several analysts triggering investigations or Dashboard executive-summary generation at the same moment — requests queue up behind one another rather than running in parallel, and per-request latency grows accordingly.

Two things are true about this limitation and worth stating plainly:

- **It never affects data correctness.** The underlying KPI numbers, deterministic risk scores, and provider evidence are always the same real values regardless of how long the AI narrative itself takes to generate — this is purely a latency characteristic of a single local model instance, not a correctness gap.
- **It does not touch the Dashboard's core KPI tiles or the Provider Health page**, both of which are pure database reads, unaffected by AI backend choice or load, and both of which were separately load-tested to 25 concurrent requests with a genuine 100%-success outcome (see `tech-09-performance.md`).

An administrator who expects many concurrent users should consider any of the ten cloud backends (Anthropic, Bedrock, Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, or OpenRouter) rather than Ollama specifically to get consistently fast AI generation under concurrent load — the runtime-switching mechanism in §5 makes this a config change, not a re-deployment.

See Also

- `tech-03-ai-architecture.md` — the architectural deep dive: the two-call structure, the full grounding/cross-check mechanism, and the `Verdict` / `RiskAssessment` data model.
- `user-07-ai-analysis.md` — the analyst-facing view: what the AI shows on an investigation page, how disagreement between providers is presented, and how to verify an AI claim against real evidence.
- `standalone-windows-installation.md` — the Setup Wizard's AI Configuration page, in the context of a full first-run installation walkthrough.
- `tech-09-performance.md` / `tech-10-limitations.md` — the broader performance and limitations picture this guide's §11 is one specific piece of.